homework1

2026-08-30

Homework 1

Question 1 (Least squares and matrix calculations)

Original question

Set the random seed to 43201. Generate $n = 100$ observations with $p = 5$ predictors according to

$$x_{ij} \overset{\text{iid}}{\sim} N(0,1), \qquad \varepsilon_i \overset{\text{iid}}{\sim} N(0, 0.7^2),$$

with all predictors and errors generated independently. Let

$$\mathbf{X} = \begin{bmatrix} \mathbf{1} & \mathbf{x}_1 & \cdots & \mathbf{x}_5 \end{bmatrix}, \qquad \beta = (3, 1.4, -0.9, 0.7, 0, 1.1)^{\mathsf{T}},$$

and generate

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon.$$

a. Generate the data and report the dimensions and rank of $\mathbf{X}$. Calculate the least-squares estimate $\widehat{\beta}$ using a numerically stable least-squares routine, without explicitly calculating $(\mathbf{X}^{\mathsf{T}}\mathbf{X})^{-1}$. Compare the estimates with the coefficients used to generate the data.

b. Calculate the fitted values, the residual vector $\mathbf{r} = \mathbf{y} - \mathbf{X}\widehat{\beta}$, and the training root mean squared error

$$\text{RMSE} = \left(\frac{1}{n} \sum_{i=1}^{n} r_i^2 \right)^{1/2}.$$

c. Report $\|\mathbf{X}^{\mathsf{T}}\mathbf{r}\|_\infty$. Explain why this value should be close to zero and what it tells us about the residual vector.

Solution

a) The design matrix has 100 rows and 6 columns: one intercept and five predictors. Its rank is 6. Using a stable least-squares method gives estimates close to the true coefficients.

For R,

$$\hat{\beta} \approx (3.006,\ 1.417,\ -0.888,\ 0.795,\ 0.048,\ 1.132)^{\mathsf{T}}.$$

For Python,

$$\hat{\beta} \approx (2.973,\ 1.324,\ -0.841,\ 0.720,\ -0.083,\ 1.165)^{\mathsf{T}}.$$

The small differences from the true values are due to random error in the simulated data.

b) The fitted values and residuals are

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\beta}, \qquad \mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}.$$

The training RMSE is about **0.6764 in R** and **0.6463 in Python**.

c) The value of

$$\|\mathbf{X}^{\mathsf{T}}\mathbf{r}\|_{\infty}$$

is about 5.9×10^{-14} in R and 1.4×10^{-12} in Python. These are essentially zero, which shows that the residuals are orthogonal to the columns of $\mathbf{X}$, as expected for least squares.

R

```r
set.seed(43201)

n <- 100
p <- 5

# Simulate the five predictors and label them.
x <- matrix(rnorm(n * p), nrow = n, ncol = p)
colnames(x) <- paste0("x", seq_len(p))

X <- cbind("(Intercept)" = 1, x)
beta <- c(
  "(Intercept)" = 3,
  x1 = 1.4,
```

```
  x2 = -0.9,
  x3 = 0.7,
  x4 = 0,
  x5 = 1.1
)

epsilon <- rnorm(n, mean = 0, sd = 0.7)
y <- as.vector(X %*% beta + epsilon)

# Fit least squares with the QR-based routine.
least_squares_fit <- lm.fit(x = X, y = y)
beta_hat <- least_squares_fit$coefficients
fitted_values <- as.vector(X %*% beta_hat)
residuals <- y - fitted_values

coefficient_comparison <- data.frame(
  coefficient = names(beta),
  truth = unname(beta),
  estimate = unname(beta_hat)
)

training_rmse <- sqrt(mean(residuals^2))
normal_equation_error <- max(abs(crossprod(X, residuals)))

print(dim(X))
print(qr(X)$rank)
print(coefficient_comparison)
print(training_rmse)
print(normal_equation_error)
```

Python

```
import numpy as np

rng = np.random.default_rng(43201)

n = 100
p = 5

# Simulate predictors and attach an intercept column.
x = rng.normal(size=(n, p))
X = np.column_stack([np.ones(n), x])
beta = np.array([3.0, 1.4, -0.9, 0.7, 0.0, 1.1])

epsilon = rng.normal(loc=0.0, scale=0.7, size=n)
y = X @ beta + epsilon
```

```python
# Use NumPy least squares for a stable fit.
beta_hat, _, rank, _ = np.linalg.lstsq(X, y, rcond=None)
fitted_values = X @ beta_hat
residuals = y - fitted_values

coefficient_comparison = np.column_stack([beta, beta_hat])
training_rmse = np.sqrt(np.mean(residuals**2))
normal_equation_error = np.max(np.abs(X.T @ residuals))

print("Dimensions:", X.shape)
print("Rank:", rank)
print("Truth and estimate:")
print(coefficient_comparison)
print("Training RMSE:", training_rmse)
print("Normal-equation error:", normal_equation_error)
```

Question 2 (Gaussian likelihood for linear regression)

Original question

Continue with the data from Question 1. Suppose

$$\mathbf{Y} \mid \mathbf{X} \sim N_n\left(\mathbf{X}\beta, \sigma^2 \mathbf{I}_n\right).$$

The log-likelihood, including the constant term, is

$$\ell(\beta, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{X}\beta)^\mathsf{T}(\mathbf{y} - \mathbf{X}\beta).$$

a. For fixed σ^2, explain why maximizing $\ell(\beta, \sigma^2)$ over β is equivalent to minimizing the residual sum of squares. What does this imply about the maximum-likelihood estimate of β?

b. For fixed β, differentiate the log-likelihood with respect to σ^2 and derive its maximum-likelihood estimate. Calculate this estimate at $\hat{\beta}$ and compare it with the unbiased estimate of σ^2. Explain why the two denominators differ.

c. Let

$$\mathbf{e}_{x_2} = (0, 0, 1, 0, 0, 0)^\mathsf{T}.$$

Using the variance estimate from part b, plot

$$\ell\left(\hat{\beta} + t\mathbf{e}_{x_2}, \hat{\sigma}^2_{\mathrm{MLE}}\right)$$

over a grid of t values from -1.25 to 1.25. State where the maximum occurs and explain why this agrees with the least-squares result.

Solution

a) With σ^2 fixed, the only part of the log-likelihood involving β is the negative RSS term. Therefore, maximizing the likelihood is the same as minimizing RSS. So,

$$\hat{\beta}_{\text{MLE}} = \hat{\beta}_{\text{LS}}.$$

b) Let $v = \sigma^2$. Then

$$\ell(\beta, v) = -\frac{n}{2}\log(2\pi v) - \frac{\text{RSS}}{2v}.$$

Differentiating and setting the result equal to zero gives

$$\hat{\sigma}^2_{\text{MLE}} = \frac{\text{RSS}}{n}.$$

The unbiased estimate instead uses

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n-6},$$

because 6 coefficients were estimated. The R values are about **0.4575** and **0.4867**; the Python values are about **0.4178** and **0.4444**, respectively.

c) The profile reaches its maximum at $t = 0$. That point is the least-squares estimate, so moving the x_2 coefficient away from it increases RSS and lowers the likelihood.

R

```r
rss <- sum(residuals^2)
sigma2_mle <- rss / n
sigma2_unbiased <- rss / (n - ncol(X))

log_likelihood <- function(beta_value, sigma2, X, y) {
  residual <- y - as.vector(X %*% beta_value)
  -nrow(X) / 2 * log(2 * pi * sigma2) -
    sum(residual^2) / (2 * sigma2)
}

e_x2 <- c(0, 0, 1, 0, 0, 0)
```

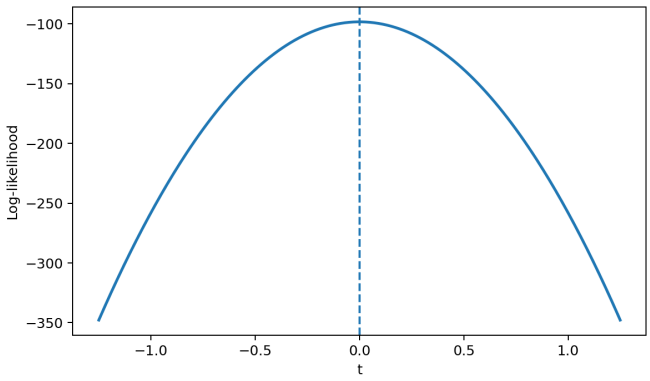

Figure 1: Likelihood profile for the coefficient of x_2.

```r
t_grid <- seq(-1.25, 1.25, length.out = 251)
profile_log_likelihood <- vapply(
  t_grid,
  function(t) {
    log_likelihood(
      beta_hat + t * e_x2,
      sigma2_mle,
      X,
      y
    )
  },
  numeric(1)
)

print(c(
  sigma2_mle = sigma2_mle,
  sigma2_unbiased = sigma2_unbiased,
  maximizing_t = t_grid[which.max(profile_log_likelihood)]
))

plot(
  t_grid,
  profile_log_likelihood,
  type = "l",
  lwd = 2,
  col = "#1F5A94",
  xlab = expression(t),
  ylab = "Log-likelihood"
)
abline(v = 0, lty = 2, lwd = 2, col = "#C84A16")
```

Python

```
import matplotlib.pyplot as plt

rss = residuals @ residuals
sigma2_mle = rss / n
sigma2_unbiased = rss / (n - X.shape[1])

def log_likelihood(beta_value, sigma2, X, y):
    residual = y - X @ beta_value
    return (
        -X.shape[0] / 2 * np.log(2 * np.pi * sigma2)
        - residual @ residual / (2 * sigma2)
    )

e_x2 = np.array([0.0, 0.0, 1.0, 0.0, 0.0, 0.0])
t_grid = np.linspace(-1.25, 1.25, 251)
profile_log_likelihood = np.array(
    [
        log_likelihood(
            beta_hat + t * e_x2,
            sigma2_mle,
            X,
            y,
        )
        for t in t_grid
    ]
)

print("MLE variance:", sigma2_mle)
print("Unbiased variance:", sigma2_unbiased)
print("Maximizing t:", t_grid[np.argmax(profile_log_likelihood)])

fig, ax = plt.subplots(figsize=(7, 4.2))
profile_line = ax.plot(
    t_grid,
    profile_log_likelihood,
    color="#1F5A94",
    linewidth=2,
)
zero_line = ax.axvline(
    0,
    color="#C84A16",
    linestyle="--",
    linewidth=2,
```

```
)
profile_labels = ax.set(xlabel=r"$t$", ylabel="Log-likelihood")
ax.spines[["top", "right"]].set_visible(False)
plt.show()
```

Question 3 (Starting values in nonconvex optimization)

Original question

Consider the function

$$f(x) = \exp(1.5x) - 3(x+6)^2 - 0.05x^3,$$

with derivative

$$f'(x) = 1.5\exp(1.5x) - 6(x+6) - 0.15x^2.$$

a. Plot $f(x)$ over the interval $[-40, 7]$. Based on the plot, describe the important features of the objective function that may affect numerical optimization.

b. Use BFGS to minimize $f(x)$ twice, starting at $x^{(0)} = -15$ and $x^{(0)} = 0$. Supply $f'(x)$ to the optimizer. For each run, report the final value of x, the final objective value, $|f'(x)|$, and whether the optimizer reported convergence.

c. Explain why the two runs can converge to different answers. Which run gives the lower objective value? What additional evidence would be needed before claiming that this point is the global minimum?

Solution

a) The graph is nonconvex: it has two local minima with a local maximum between them. Because of this, a gradient-based method can end at different points depending on its starting value.

b) Starting at -15 gives approximately

$$x = -32.64911, \qquad f(x) = -390.38577.$$

Starting at 0 gives approximately

$$x = 2.34997, \qquad f(x) = -175.86262.$$

Both runs converge and have gradients very close to zero.

c) The start at -15 finds the lower objective value. Still, that alone does not prove it is the global minimum. We would need to check all stationary points and the behavior of $f(x)$ over the full domain.

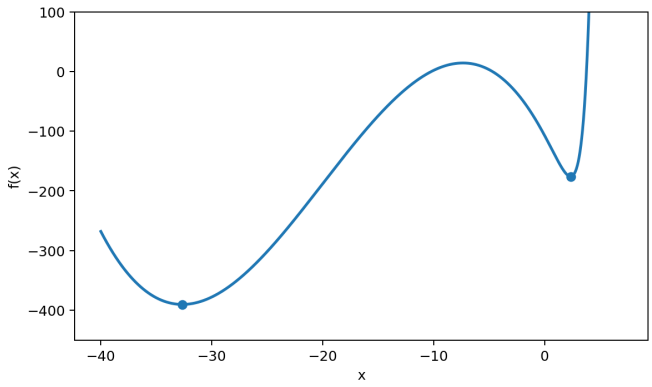

Figure 2: Nonconvex objective with the two BFGS solutions.

R

```r
objective <- function(x) {
  exp(1.5 * x) - 3 * (x + 6)^2 - 0.05 * x^3
}

gradient <- function(x) {
  1.5 * exp(1.5 * x) - 6 * (x + 6) - 0.15 * x^2
}

x_grid <- seq(-40, 7, length.out = 1200)

# Limit the plotted y-range so both minima can be seen.
plot(
  x_grid,
  objective(x_grid),
  type = "l",
  lwd = 2,
  col = "#13294B",
  ylim = c(-450, 100),
  xlab = "x",
  ylab = "f(x)"
)

starts <- c(-15, 0)
optimization_results <- lapply(
```

```r
  starts,
  function(start) {
    optim(
      par = start,
      fn = objective,
      gr = gradient,
      method = "BFGS"
    )
  }
)

optimization_summary <- do.call(
  rbind,
  Map(
    function(start, result) {
      data.frame(
        starting_value = start,
        final_x = result$par,
        objective = result$value,
        absolute_gradient = abs(gradient(result$par)),
        converged = result$convergence == 0
      )
    },
    starts,
    optimization_results
  )
)

points(
  optimization_summary$final_x,
  optimization_summary$objective,
  pch = 19,
  col = c("#2F6FB3", "#C84A16")
)

print(optimization_summary)
```

Python

```python
from scipy.optimize import minimize

def objective(x):
    return np.exp(1.5 * x) - 3 * (x + 6) ** 2 - 0.05 * x**3

def gradient(x):
    return 1.5 * np.exp(1.5 * x) - 6 * (x + 6) - 0.15 * x**2
```

```
x_grid = np.linspace(-40, 7, 1200)

fig, ax = plt.subplots(figsize=(7, 4.2))
objective_line = ax.plot(
    x_grid,
    objective(x_grid),
    color="#13294B",
    linewidth=2,
)
objective_labels = ax.set(
    ylim=(-450, 100),
    xlabel=r"$x$",
    ylabel=r"$f(x)$",
)
ax.spines[["top", "right"]].set_visible(False)

starts = [-15.0, 0.0]
optimization_results = []
for start in starts:
    result = minimize(
        fun=lambda value: objective(value[0]),
        x0=np.array([start]),
        jac=lambda value: np.array([gradient(value[0])]),
        method="BFGS",
    )
    optimization_results.append(result)

optimization_summary = []
for start, result in zip(starts, optimization_results):
    optimization_summary.append(
        {
            "starting_value": start,
            "final_x": result.x[0],
            "objective": result.fun,
            "absolute_gradient": abs(gradient(result.x[0])),
            "converged": result.success,
        }
    )

minimum_points = ax.scatter(
    [result.x[0] for result in optimization_results],
    [result.fun for result in optimization_results],
    color=["#2F6FB3", "#C84A16"],
    zorder=3,
)
```

```
plt.show()

for row in optimization_summary:
    print(
        f"start={row['starting_value']:5.1f}, "
        f"x={float(row['final_x']): .6f}, "
        f"f(x)={float(row['objective']): .6f}, "
        f"|f'(x)|={float(row['absolute_gradient']):.3e}, "
        f"converged={row['converged']}"
    )
```

Question 4 (Nearly collinear predictors)

Original question

Continue with $\mathbf{X}$ and $\mathbf{y}$ from Question 1. Set the random seed to 43202 and generate

$$u_i \overset{\text{iid}}{\sim} N(0,1), \qquad i = 1, \dots, n.$$

Define

$$\mathbf{x}_6 = \mathbf{x}_1 + 10^{-4}\mathbf{u}, \qquad \mathbf{X}_+ = \begin{bmatrix} \mathbf{X} & \mathbf{x}_6 \end{bmatrix},$$

and let

$$\mathbf{y}^\star = \mathbf{y} + 10^{-3}\mathbf{u}.$$

Use a numerically stable least-squares routine throughout.

a. Fit $\mathbf{y}$ using $\mathbf{X}$ and $\mathbf{X}_+$. For each design matrix, report the condition number and training RMSE. For the augmented fit, also report the coefficients of $\mathbf{x}_1$ and $\mathbf{x}_6$.

b. Fit $\mathbf{y}^\star$ using $\mathbf{X}_+$. Report the changes in the coefficients of $\mathbf{x}_1$ and $\mathbf{x}_6$. Compare the two fitted-value vectors using their root mean squared difference and maximum absolute difference.

c. Use the identity

$$\mathbf{y}^\star - \mathbf{y} = 10(\mathbf{x}_6 - \mathbf{x}_1)$$

to explain why the fitted values change very little while the two coefficients change substantially. What does this example suggest about interpreting separate effects for nearly collinear predictors?

Solution

a) Adding x_6 makes the predictors nearly collinear because

$$\mathbf{x}_6 - \mathbf{x}_1 = 10^{-4}\mathbf{u}.$$

The condition number rises from about 1.5 to about 2×10^4, while the training RMSE barely changes. The two nearly duplicate predictors get large opposite coefficients:

Implementation	$\hat{\beta}_1$	$\hat{\beta}_6$	Sum
R	351.677	-350.259	1.418
Python	-1329.743	1331.060	1.317

The exact values differ across R and Python because their random draws are different, but the same instability appears in both.

b) Since

$$\mathbf{y}^\star - \mathbf{y} = 10(\mathbf{x}_6 - \mathbf{x}_1),$$

the change can be represented by roughly

$$\Delta\hat{\beta}_1 = -10, \qquad \Delta\hat{\beta}_6 = 10.$$

The fitted values change very little: the RMSE difference is about **0.00102 in R** and **0.000975 in Python**, with maximum differences about **0.00257** and **0.00235**.

c) Nearly collinear variables can have unstable individual coefficients even when predictions stay almost the same. Their combined effect is much more reliable than interpreting each coefficient separately.

R

```
set.seed(43202)
u <- rnorm(n)

x6 <- x[, "x1"] + 1e-4 * u
X_plus <- cbind(X, x6 = x6)
y_star <- y + 1e-3 * u

fit_original <- lm.fit(x = X, y = y)
fit_augmented <- lm.fit(x = X_plus, y = y)
```

```r
fit_perturbed <- lm.fit(x = X_plus, y = y_star)

beta_augmented <- fit_augmented$coefficients
beta_perturbed <- fit_perturbed$coefficients

fitted_augmented <- as.vector(X_plus %*% beta_augmented)
fitted_perturbed <- as.vector(X_plus %*% beta_perturbed)

condition_number <- function(design) {
  singular_values <- svd(design, nu = 0, nv = 0)$d
  max(singular_values) / min(singular_values)
}

fit_summary <- data.frame(
  design = c("X", "X_plus"),
  condition_number = c(
    condition_number(X),
    condition_number(X_plus)
  ),
  training_rmse = c(
    sqrt(mean(fit_original$residuals^2)),
    sqrt(mean(fit_augmented$residuals^2))
  )
)

coefficient_summary <- data.frame(
  response = c("y", "y_star"),
  beta_x1 = c(beta_augmented["x1"], beta_perturbed["x1"]),
  beta_x6 = c(beta_augmented["x6"], beta_perturbed["x6"])
)

coefficient_change <- (
  beta_perturbed[c("x1", "x6")] -
    beta_augmented[c("x1", "x6")]
)
fitted_difference <- fitted_perturbed - fitted_augmented

print(fit_summary)
print(coefficient_summary)
print(coefficient_change)
print(c(
  fitted_difference_rmse = sqrt(mean(fitted_difference^2)),
  fitted_difference_max = max(abs(fitted_difference))
))
```

Python

```
rng_q4 = np.random.default_rng(43202)
u = rng_q4.normal(size=n)

x6 = x[:, 0] + 1e-4 * u
X_plus = np.column_stack([X, x6])
y_star = y + 1e-3 * u

beta_original = np.linalg.lstsq(X, y, rcond=None)[0]
beta_augmented = np.linalg.lstsq(X_plus, y, rcond=None)[0]
beta_perturbed = np.linalg.lstsq(
    X_plus, y_star, rcond=None
)[0]

fitted_original = X @ beta_original
fitted_augmented = X_plus @ beta_augmented
fitted_perturbed = X_plus @ beta_perturbed

condition_X = np.linalg.cond(X)
condition_X_plus = np.linalg.cond(X_plus)
rmse_X = np.sqrt(np.mean((y - fitted_original) ** 2))
rmse_X_plus = np.sqrt(
    np.mean((y - fitted_augmented) ** 2)
)

coefficient_change = (
    beta_perturbed[[1, 6]] - beta_augmented[[1, 6]]
)
fitted_difference = fitted_perturbed - fitted_augmented

print("X condition number and RMSE:", condition_X, rmse_X)
print(
    "X_plus condition number and RMSE:",
    condition_X_plus,
    rmse_X_plus,
)
print("Coefficients for y:", beta_augmented[[1, 6]])
print("Coefficients for y_star:", beta_perturbed[[1, 6]])
print("Coefficient changes:", coefficient_change)
print(
    "Fitted-value difference RMSE:",
    np.sqrt(np.mean(fitted_difference**2)),
)
print(
    "Fitted-value maximum difference:",
    np.max(np.abs(fitted_difference)),
```

)

Question 5 (Data preparation and summary statistics)

Original question

The file `data/data-manipulation.csv` is a small constructed data table containing an observation identifier, a group label, two numeric predictors, and a response. Some response values are missing. For analyses involving the response, use only observations with a recorded response. Do not replace a missing response with zero or a group mean.

a. Read the data and report its dimensions, column types, number of duplicated identifiers, and number of missing values in each column. Create an analysis table containing only observations with a recorded response, and define

$$\texttt{feature\_sum} = \texttt{x1} + \texttt{x2}.$$

b. For each group, report the number of observations, the mean response, and the mean of `feature_sum`. State clearly which observations these summaries describe.

c. Sort the analysis table by response from largest to smallest and report the first three rows, including `observation_id`, `group`, `response`, and `feature_sum`. Add code checks verifying that the number of retained rows equals the number of nonmissing responses in the original data, that the analysis table has no missing response values, and that its identifiers are unique.

Solution

a) The data have **24 rows and 5 columns**. `observation_id` and `group` are text; `x1`, `x2`, and `response` are numeric. There are no duplicate IDs. Only `response` has missing values, with **3 missing**, so the analysis table has **21 rows** after removing them.

b) Using only rows with a recorded response:

Group	Observations	Mean response	Mean `feature_sum`
A	7	5.6000	3.6000
B	7	7.6029	4.9286
C	7	7.9971	6.5857

c) The three highest responses are:

observation_id	Group	Response	feature_sum
obs-22	C	9.24	6.6
obs-16	B	9.00	5.2
obs-14	B	8.66	5.0

The code checks confirm that all retained responses are nonmissing, the row count matches the number of observed responses, and the IDs are unique.

R

```
# Find the data file from either working location.
data_file <- file.path("data", "data-manipulation.csv")
if (!file.exists(data_file)) {
  document_dir <- Sys.getenv("QUARTO_DOCUMENT_PATH", unset = ".")
  data_file <- file.path(
    document_dir,
    "data",
    "data-manipulation.csv"
  )
}
small_data <- read.csv(
  data_file,
  stringsAsFactors = FALSE,
  check.names = FALSE
)

print(dim(small_data))
print(vapply(small_data, class, character(1)))
print(c(
  duplicated_identifiers =
    sum(duplicated(small_data$observation_id))
))
print(colSums(is.na(small_data)))

# Use complete response rows for the analysis.
analysis_data <- small_data[
  !is.na(small_data$response),
]
analysis_data$feature_sum <- (
  analysis_data$x1 + analysis_data$x2
)

group_summary <- do.call(
  rbind,
```

```
  lapply(
    split(analysis_data, analysis_data$group),
    function(group_data) {
      data.frame(
        group = group_data$group[1],
        observations = nrow(group_data),
        mean_response = mean(group_data$response),
        mean_feature_sum = mean(group_data$feature_sum),
        row.names = NULL
      )
    }
  )
)

largest_three <- analysis_data[
  order(analysis_data$response, decreasing = TRUE),
  c("observation_id", "group", "response", "feature_sum")
][1:3, ]

number_recorded <- sum(!is.na(small_data$response))
stopifnot(
  nrow(analysis_data) == number_recorded,
  !anyNA(analysis_data$response),
  !anyDuplicated(analysis_data$observation_id)
)

print(group_summary)
print(largest_three)
```

Python

```
import os
from pathlib import Path

import pandas as pd

# Find the data file from either working location.
data_file = Path("data") / "data-manipulation.csv"
if not data_file.exists():
    document_dir = Path(
        os.environ.get("QUARTO_DOCUMENT_PATH", ".")
    )
    data_file = document_dir / "data" / "data-manipulation.csv"
small_data = pd.read_csv(data_file)

print("Dimensions:", small_data.shape)
```

```
print(small_data.dtypes)
print(
    "Duplicated identifiers:",
    small_data["observation_id"].duplicated().sum(),
)
print(small_data.isna().sum())

# Use complete response rows for the analysis.
analysis_data = (
    small_data.dropna(subset=["response"])
    .copy()
)
analysis_data["feature_sum"] = (
    analysis_data["x1"] + analysis_data["x2"]
)

group_summary = (
    analysis_data.groupby("group", as_index=False)
    .agg(
        observations=("response", "size"),
        mean_response=("response", "mean"),
        mean_feature_sum=("feature_sum", "mean"),
    )
)

largest_three = (
    analysis_data.sort_values("response", ascending=False)
    .loc[
        :,
        [
            "observation_id",
            "group",
            "response",
            "feature_sum",
        ],
    ]
    .head(3)
)

number_recorded = small_data["response"].notna().sum()
assert len(analysis_data) == number_recorded
assert not analysis_data["response"].isna().any()
assert analysis_data["observation_id"].is_unique

print(group_summary.to_string(index=False))
print(largest_three.to_string(index=False))
```

Question 6 (Can 39 million Fitbit records represent US adults?)

Original question

Patten et al. (2026) describe Fitbit data from 59,018 participants in the All of Us Research Program. The dataset spans 14 years and contains more than 39 million daily step records. Participants contributed data through one of two routes:

- Bring Your Own Device (BYOD): participants shared data from a Fitbit they already owned.
- Wearables Enhancing All of Us Research (WEAR): invited participants received a Fitbit at no cost.

In the general activity cohort, the BYOD and WEAR groups contained 32,035 and 22,474 participants, respectively. The BYOD value is listed first in each comparison below:

- 77.3% versus 55.1% reported being White;
- 6.1% versus 15.2% reported annual household income between $10,000 and $25,000; and
- median daily steps were 6,867 versus 5,797.

Suppose the target is the mean of the participant-specific average daily step counts among US adults during the study period, with each adult given equal weight. A researcher writes:

> "This dataset contains more than 39 million daily Fitbit records. Therefore, the average of all recorded step counts should provide an accurate estimate of mean daily activity among US adults."

a. Are the 39 million daily records independent observations? What characteristics or behaviors could cause some participants to contribute more recorded days than others? Explain how averaging all recorded days would then weight participants unequally.

b. BYOD participants reported a median of 6,867 daily steps, compared with 5,797 among WEAR participants. Does this comparison show that already owning a Fitbit causes people to walk more? Explain your answer and give at least one plausible alternative explanation based on how participants entered the two groups.

c. A very large dataset can still produce a biased estimate of a population quantity when the people and observations entering the dataset are selected. Using this study, explain how selection of both participants and recorded days could make the observed data differ from the US adult population. Why might the naive average of all recorded daily step counts fail to estimate the stated target? You do not need to determine the direction of the bias.

Source

Patten, T., Preble, E. A., Master, H., et al. (2026). The All of Us Research Program's wearables dataset. *Nature Medicine*, 32, 2302-2310.

Solution

a) No. The 39 million rows are repeated days from about 59,000 people, so observations from the same person are related. People also contribute different numbers of days because of enrollment length, device use, syncing, missing days, travel, illness, or device problems. Averaging every day therefore gives more weight to people with more recorded days.

b) No causal conclusion can be made. BYOD and WEAR were not randomly assigned. People who already owned a Fitbit may differ in income, technology access, health interest, or activity level. The demographic differences between the groups also show that they are not directly comparable.

c) The sample can differ from US adults at several points: joining All of Us, entering BYOD or WEAR, agreeing to share data, connecting the device, and continuing to record valid days. A huge number of records lowers random error but does not remove selection bias.

For the stated target, a better approach is to first calculate each participant's average steps so each person has equal weight, then consider population weighting or calibration. Even then, unmeasured selection may remain.